

StatusNest

Solution Architecture Document

Document Version: 1.0

Author: Muhammad Abdullah

Date: July 2026

Status: Final

Table of Contents

Table of Contents	2
1. Executive Summary	5
2. Project Overview	5
2.1 Problem Statement	5
2.2 Solution Summary	5
2.3 Repositories	5
3. Stakeholders & Goals.....	6
3.1 Stakeholders	6
3.2 Functional Requirements	6
3.3 Non-Functional Requirements	6
4. Architecture Overview.....	7
4.1 High-Level Architecture	7
4.2 Request Flow — Dashboard	7
4.3 Request Flow — Monitoring.....	7
5. Component Architecture.....	8
5.1 Frontend — statusnest-frontend	8
Pages.....	8
5.2 Auth Service — Port 8000	8
Endpoints.....	8
5.3 Monitor Service — Port 8001.....	9
Endpoints.....	9
5.4 Status Service — Port 8002	9
Endpoints.....	9
Redis-first pattern.....	10
5.5 Monitor Worker Lambda	10
5.6 Processor Lambda	11
6. Data Architecture	12
6.1 PostgreSQL Schema (RDS)	12
6.2 Redis Cache Schema	12
6.3 Data Flow	13
7. Security Architecture.....	14
7.1 Layers of Defense	14
7.2 Authentication & Authorization	14
7.3 Network Security.....	15
7.4 Secrets Management.....	15

7.5 CI/CD Security — OIDC	15
8. Observability & Monitoring	16
8.1 CloudWatch Dashboard — statusnest-dev	16
8.2 CloudWatch Alarms	16
8.3 AWS X-Ray Tracing	16
8.4 Structured Logging	16
9. CI/CD Pipeline	17
9.1 Pipeline Overview	17
9.2 Environments	17
9.3 Docker Build	17
10. Infrastructure as Code	18
10.1 Terraform Module Structure	18
10.2 Remote State	18
10.3 Deployment	18
11. Architectural Decisions (ADRs)	18
ADR-001: ECS Fargate over EC2	18
ADR-002: Lambda + SQS for Monitoring Engine	19
ADR-003: Redis as Primary Status Cache	19
ADR-004: CloudFront as Single Entry Point	19
ADR-005: PostgreSQL over Aurora	19
ADR-006: Soft Delete for Services	19
ADR-007: OIDC over IAM Access Keys for CI/CD	19
ADR-008: pg8000 over psycopg2 for Lambda	19
12. Trade-offs & Constraints	20
13. Scalability & Performance	20
13.1 Horizontal Scaling	20
13.2 Performance Targets	20
13.3 Bottlenecks at Scale	21
14. Cost Considerations	21
14.1 Estimated Monthly Cost (Dev Environment)	21
14.2 Cost Optimization Opportunities	21
15. Known Limitations & Future Work	22
15.1 Known Limitations	22
15.2 Roadmap (Future Work)	22
Phase 2 — Multi-tenancy	22
Phase 3 — Notifications	22
Phase 4 — Production Hardening	22

Phase 5 — Analytics..... 22

Appendix A — Technology Stack Summary..... 23

1. Executive Summary

StatusNest is a cloud-native, multi-tenant SaaS platform that enables developers to monitor their web services and publish real-time public status pages. It is built entirely on AWS using a microservices architecture, serverless compute, managed data stores, and a globally distributed CDN.

The platform monitors registered services every 60 seconds via AWS Lambda, caches results in Redis for low-latency reads, persists historical data in PostgreSQL, and serves a React frontend through CloudFront with WAF protection.

This document describes the architecture, design decisions, trade-offs, and infrastructure of StatusNest as implemented in its MVP release.

2. Project Overview

2.1 Problem Statement

Developers and small teams need a simple, reliable way to:

- Know when their services go down, before their users do
- Communicate outages transparently via a public status page
- Track historical uptime and response times

Existing solutions (Statuspage.io, Better Uptime) are expensive and opaque. StatusNest is a self-hosted, fully open-source alternative built on AWS with production-grade infrastructure.

2.2 Solution Summary

Aspect	Detail
Platform	AWS (us-east-1)
Architecture style	Microservices + Serverless
Frontend	React SPA on S3 + CloudFront
Backend	3 FastAPI services on ECS Fargate
Monitoring engine	AWS Lambda + SQS
Data stores	RDS PostgreSQL + ElastiCache Redis
IaC	Terraform (modular)
CI/CD	GitHub Actions with OIDC
Security	WAF v2, JWT auth, HTTPS enforced

2.3 Repositories

Repo	Purpose
statusnest-frontend	React SPA

Repo	Purpose
statusnest-api	FastAPI microservices
statusnest-worker	Lambda monitoring engine
statusnest-infra	Terraform IaC

3. Stakeholders & Goals

3.1 Stakeholders

Role	Concern
End User (Developer)	Register services, view dashboard, share status page
Public Visitor	View real-time status of a developer's services
Platform Owner	Reliability, cost, security, observability

3.2 Functional Requirements

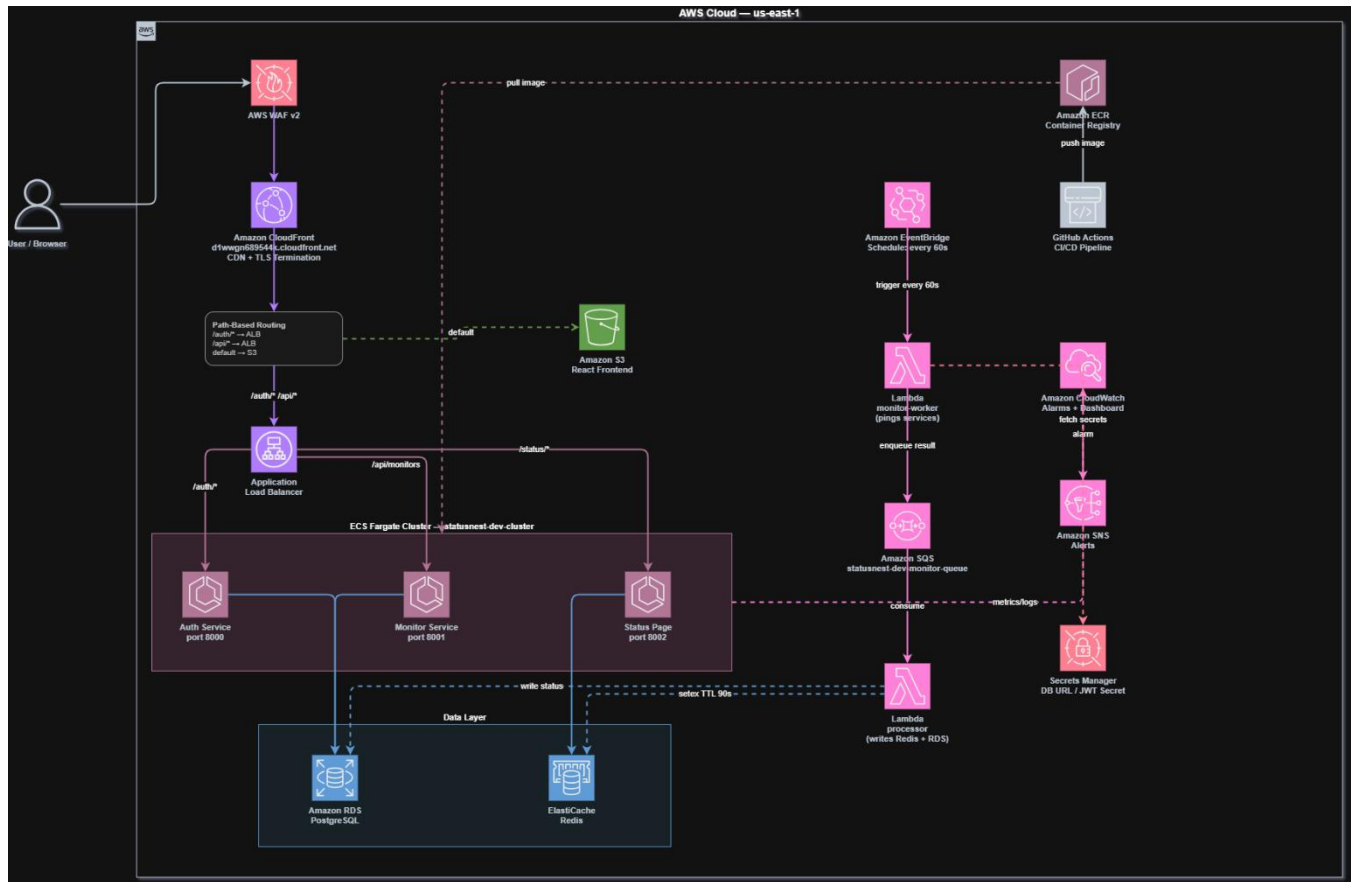
- Users can register, log in, and manage their account
- Users can add, view, and delete monitored services
- Services are pinged every 60 seconds automatically
- Current status and response time are displayed in real time
- Users can create and update incidents
- Users can add email subscribers
- Each user gets a public status page at `/status/:username`
- Public status page requires no authentication

3.3 Non-Functional Requirements

Requirement	Target
Availability	99.9% uptime
Monitoring frequency	Every 60 seconds
Status read latency	< 100ms (Redis cache)
Security	HTTPS enforced, WAF, JWT auth
Scalability	Stateless services, horizontal scaling
Observability	CloudWatch dashboards + alarms
IaC coverage	100% Terraform-managed infrastructure

4. Architecture Overview

4.1 High-Level Architecture



4.2 Request Flow — Dashboard

1. Browser hits `https://d1wwgn689544k.cloudfront.net`
2. WAF v2 inspects and allows request
3. CloudFront routes default traffic to S3 (React SPA)
4. React app makes API calls to `/auth/*` and `/api/*`
5. CloudFront routes these to ALB then ECS Fargate services
6. Auth service validates JWT; monitor/status services query Redis or RDS

4.3 Request Flow — Monitoring

1. EventBridge rule fires every 60 seconds
2. monitor-worker Lambda queries RDS for all active services
3. Lambda pings each URL (HTTP GET, 10s timeout)
4. Result `{service_id, status, latency_ms}` sent to SQS
5. processor Lambda reads SQS messages
6. Writes `{status, response_time, checked_at}` to Redis (TTL 90s)

- 7. Inserts row into service_status table in RDS

5. Component Architecture

5.1 Frontend — statusnest-frontend

Aspect	Detail
Framework	React 18 with React Router v6
Build	Create React App, static assets
Hosting	AWS S3 (private bucket, no static website hosting)
Distribution	CloudFront with OAC (Origin Access Control)
TLS	CloudFront default certificate (HTTPS enforced)

Pages

Page	Route	Auth Required
Landing	/	No
Login	/login	No
Dashboard	/dashboard	Yes (JWT)
Public Status	/status/:username	No

Key design decisions:

- All API calls go through CloudFront (not directly to ALB) to avoid mixed-content and CORS issues
- React Router is supported via CloudFront custom error responses (403/404 to /index.html)
- No server-side rendering, pure SPA

5.2 Auth Service — Port 8000

Handles user identity. Runs as an ECS Fargate task.

Endpoints

Method	Path	Description
POST	/auth/register	Create user account
POST	/auth/login	Returns JWT access token
GET	/auth/me	Current user info
GET	/health	ALB health check

Implementation:

- Passwords hashed with bcrypt (pinned to 4.0.1 for passlib compatibility)
- JWT tokens signed with HS256, 24h expiry
- SQLAlchemy ORM with PostgreSQL
- X-Ray tracing enabled

5.3 Monitor Service — Port 8001

Handles all authenticated CRUD operations. Runs as an ECS Fargate task.

Endpoints

Method	Path	Description
GET/POST	/api/monitor/services	List / add services
DELETE	/api/monitor/services/{id}	Soft-delete service
GET/POST	/api/monitor/incidents	List / create incidents
PATCH	/api/monitor/incidents/{id}	Update incident status
GET/POST	/api/monitor/subscribers	List / add subscribers
GET	/health	ALB health check

Implementation:

- All endpoints require Bearer JWT token
- Service deletion is a soft-delete (`is_active = false`)
- X-Ray tracing enabled

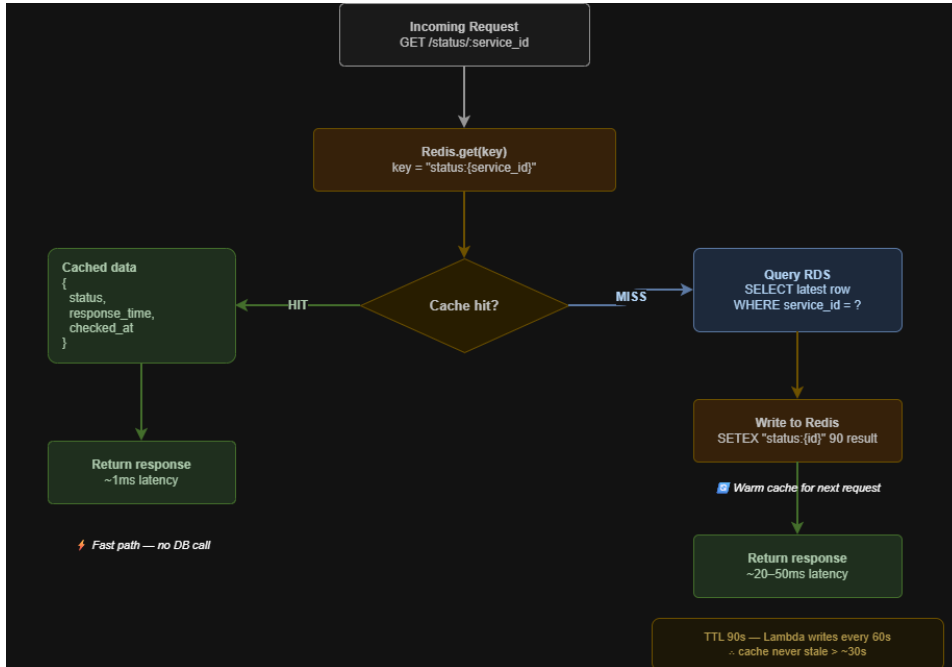
5.4 Status Service — Port 8002

Public read-only service. No authentication required.

Endpoints

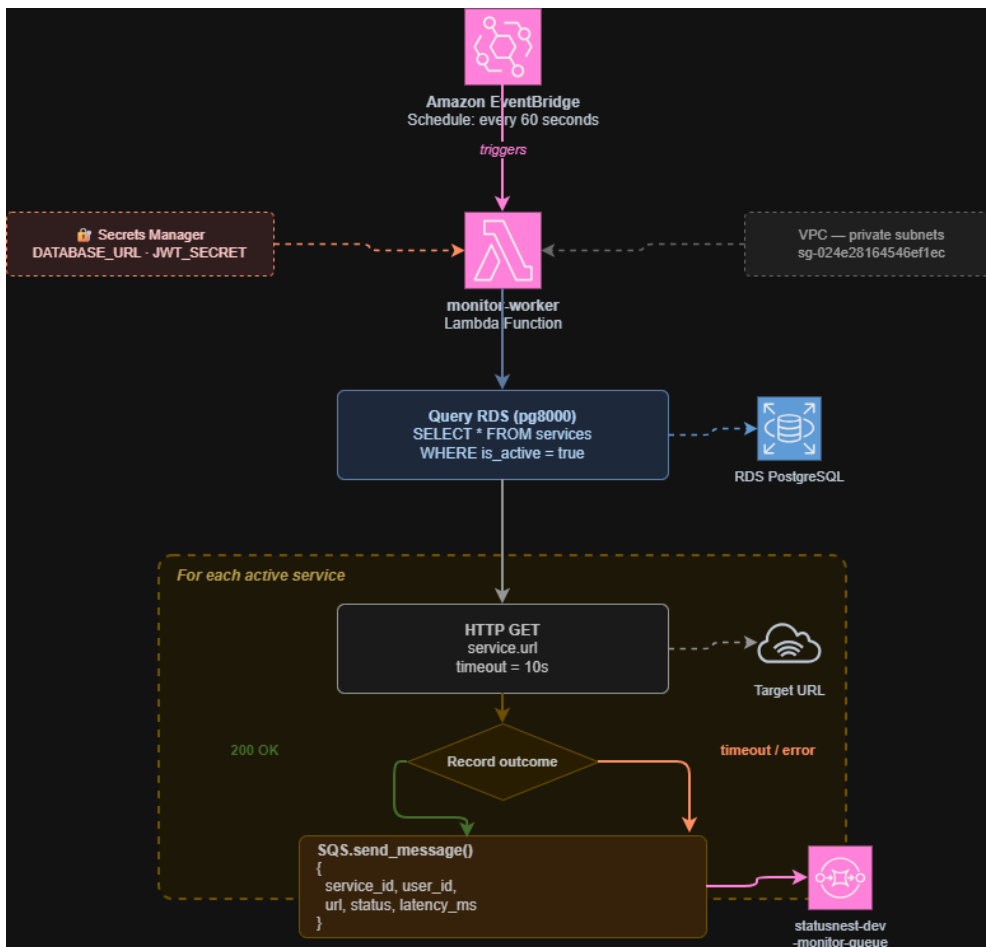
Method	Path	Description
GET	/api/status/{username}	Full status page data
GET	/api/status/{username}/history/{service_id}	24h history
GET	/health	ALB health check

Redis-first pattern



5.5 Monitor Worker Lambda

Triggered by EventBridge every 60 seconds.



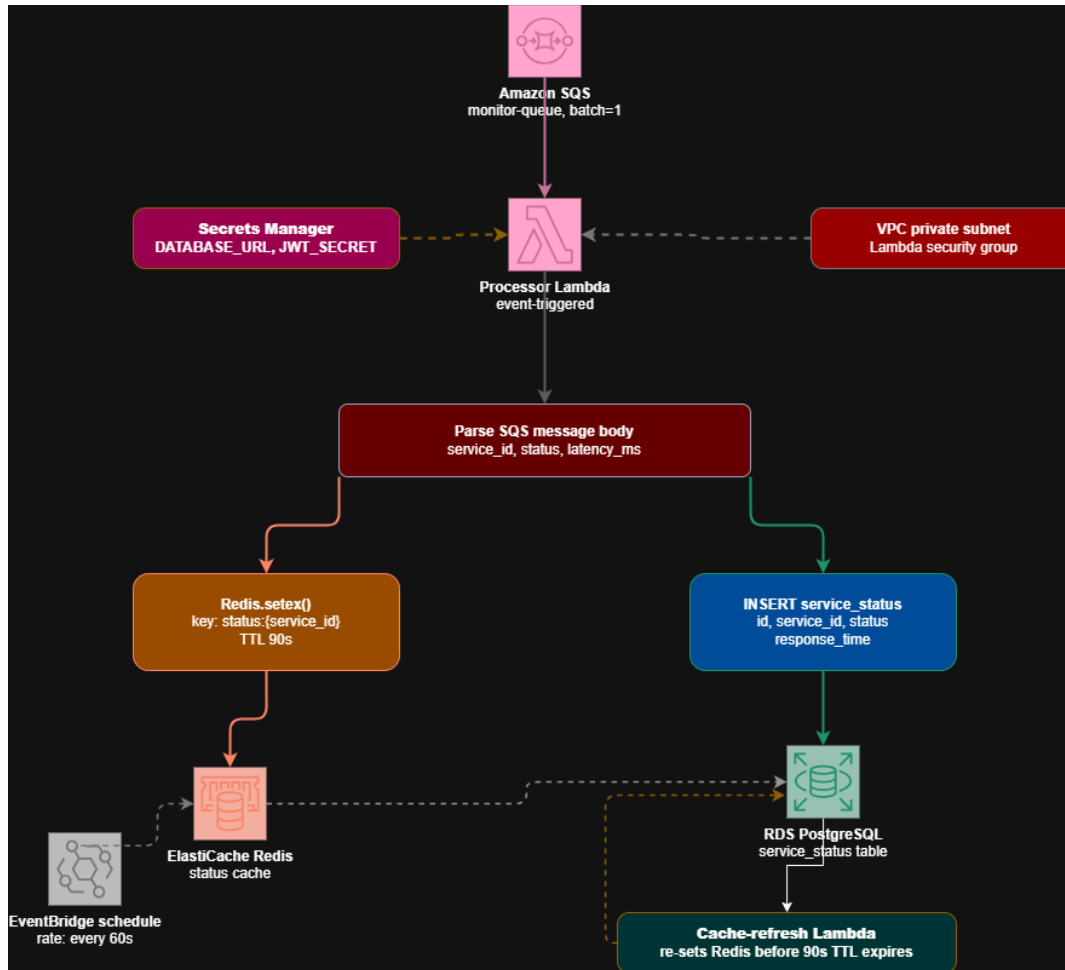
Runtime: Python 3.11

Dependencies: pg8000 (pure-Python, no native binaries), boto3

Packaging: Zipped with dependencies for Lambda deployment

5.6 Processor Lambda

Triggered by SQS queue (batch size: 1).



Runtime: Python 3.11

Dependencies: redis-py, pg8000, boto3

Dead Letter Queue: statusnest-dev-monitor-dlq (failed messages after 3 retries)

6. Data Architecture

6.1 PostgreSQL Schema (RDS)

```
-- Users
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR UNIQUE NOT NULL,
  password    VARCHAR NOT NULL,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Services
CREATE TABLE services (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  name        VARCHAR NOT NULL,
  url         VARCHAR NOT NULL,
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Service Status History
CREATE TABLE service_status (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  service_id  UUID REFERENCES services(id),
  status      VARCHAR NOT NULL, -- 'UP' | 'DOWN'
  response_time FLOAT,
  checked_at  TIMESTAMP DEFAULT NOW()
);

-- Incidents
CREATE TABLE incidents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  service_id  UUID REFERENCES services(id),
  title       VARCHAR NOT NULL,
  description  TEXT,
  status      VARCHAR DEFAULT 'investigating',
  created_at  TIMESTAMP DEFAULT NOW(),
  updated_at  TIMESTAMP DEFAULT NOW()
);

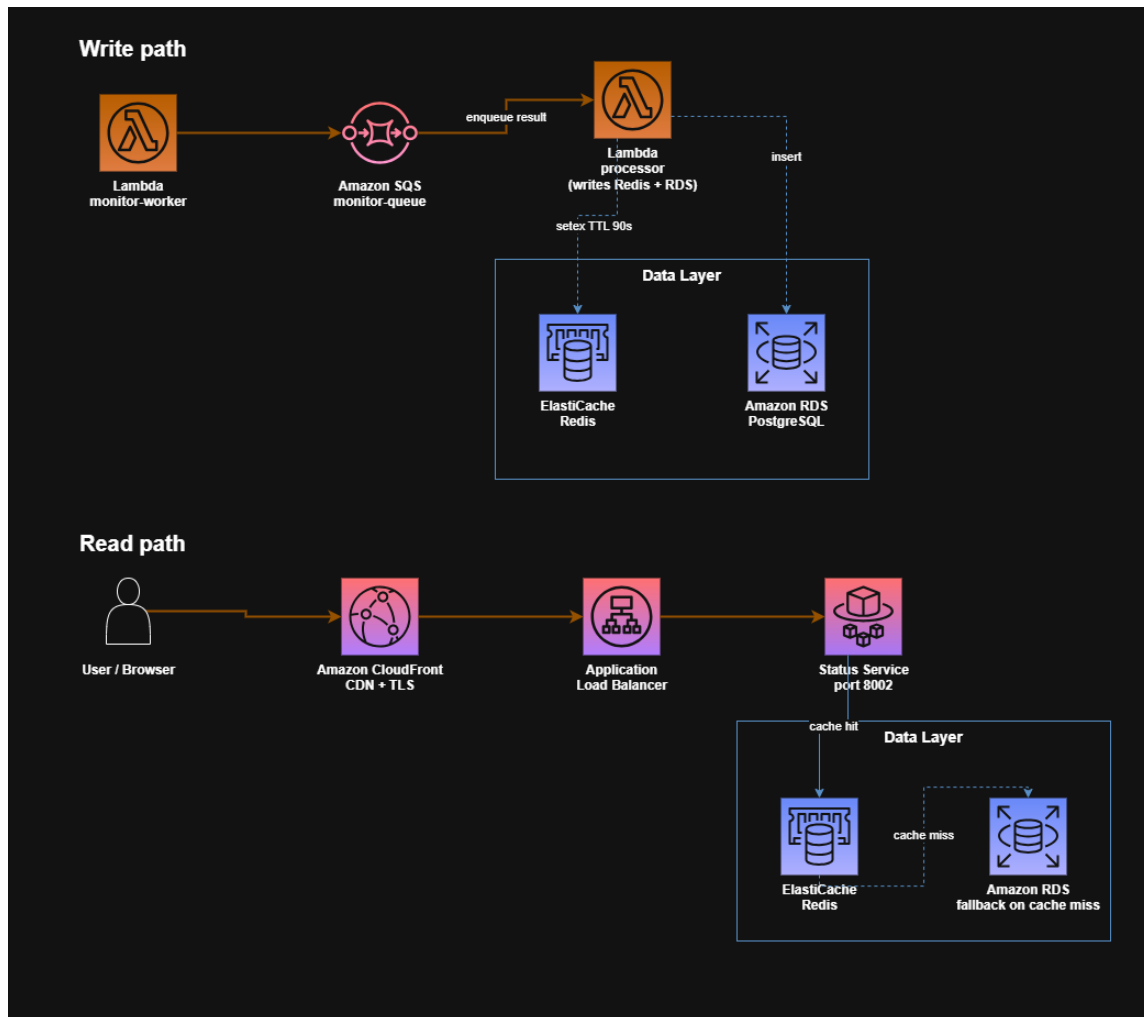
-- Subscribers
CREATE TABLE subscribers (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  email       VARCHAR NOT NULL,
  created_at  TIMESTAMP DEFAULT NOW()
);
```

6.2 Redis Cache Schema

Key Pattern	Value	TTL
status:{service_id}	{"status": "UP", "response_time": 42.3, "checked_at": "2026-07-03T..."}	90s

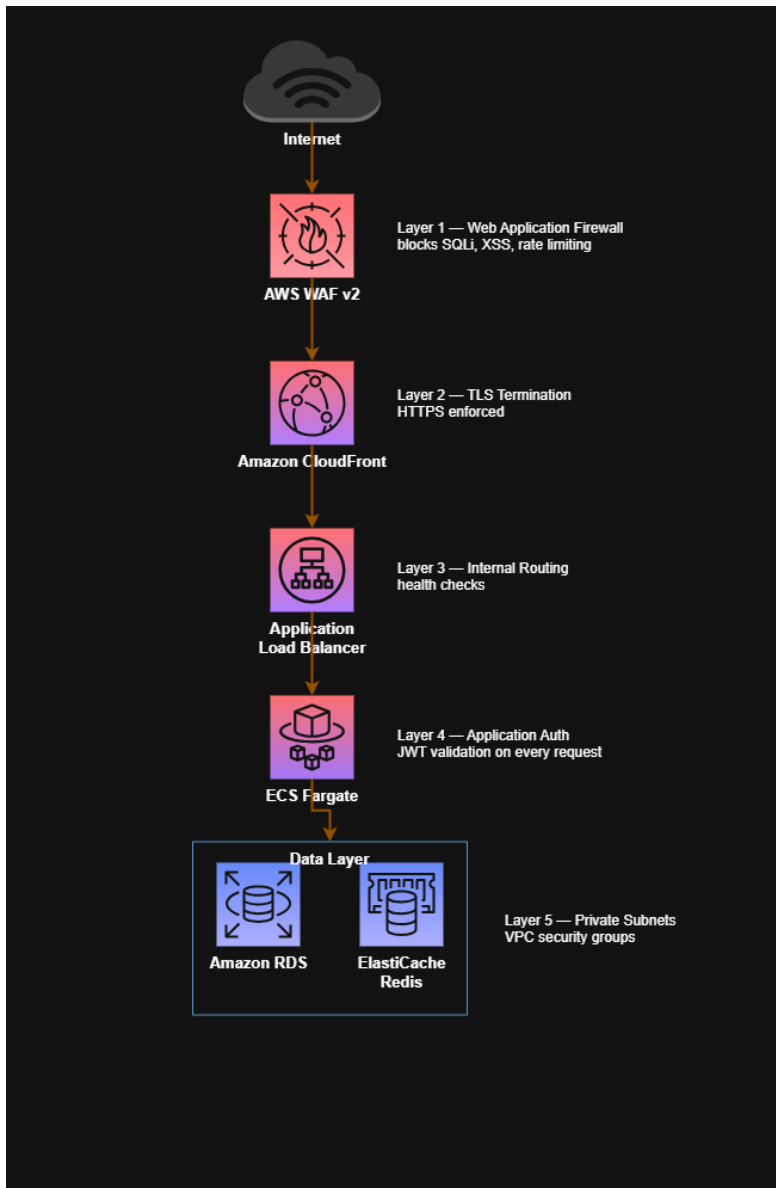
Rationale for 90s TTL: Monitor fires every 60s. A 90s TTL ensures the cache is always warm for active services while expiring stale data if monitoring stops.

6.3 Data Flow



7. Security Architecture

7.1 Layers of Defense



7.2 Authentication & Authorization

- JWT tokens (HS256) issued by auth service on login
- Token expiry: 24 hours
- All monitor service endpoints require Authorization: Bearer <token>
- Status service endpoints are intentionally public (no auth)
- Passwords hashed with bcrypt (cost factor 12)

7.3 Network Security

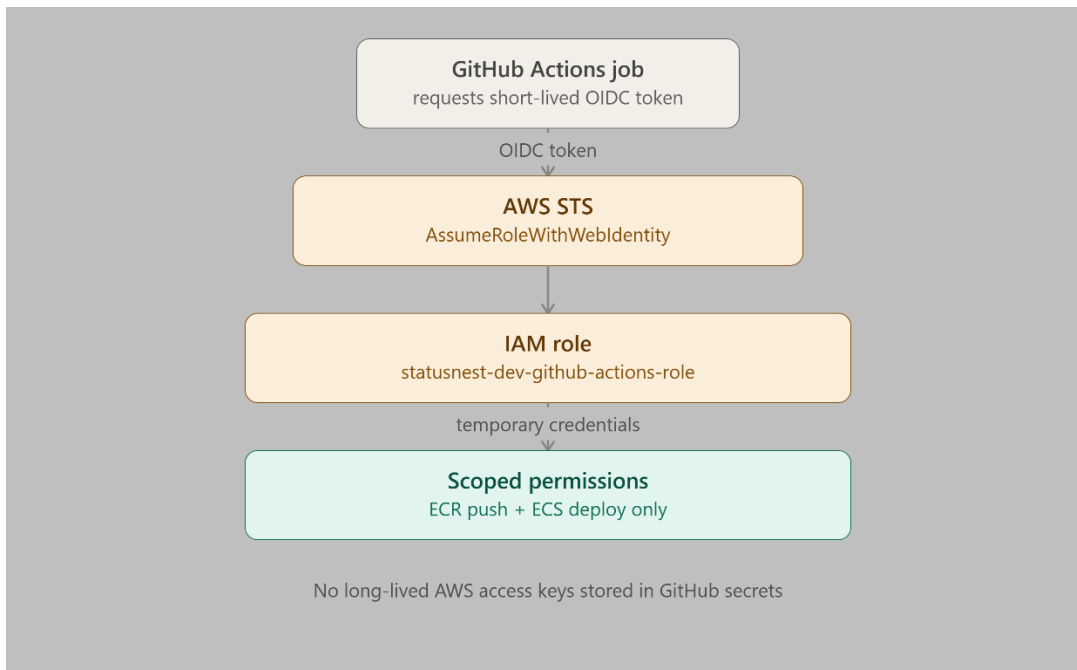
Resource	Access
RDS	Private subnet only, SG allows only ECS tasks
Redis	Private subnet only, SG allows only ECS tasks + Lambda
ALB	Public subnet, SG allows 80/443 from CloudFront only
ECS Tasks	Private subnet, outbound via NAT Gateway
Lambda	VPC-attached, private subnet

7.4 Secrets Management

- Database credentials stored in AWS Secrets Manager
- Injected into ECS task definitions as environment variables at runtime
- No secrets in code or GitHub

7.5 CI/CD Security — OIDC

GitHub Actions uses OpenID Connect (OIDC) to assume an IAM role, with no long-lived AWS access keys stored in GitHub secrets.



8. Observability & Monitoring

8.1 CloudWatch Dashboard — statusnest-dev

Widget	Metrics
ECS CPU Utilization	Per service (auth, monitor, status)
ECS Memory Utilization	Per service
ALB 5XX Errors	HTTPCode_ELB_5XX_Count
ALB Request Count	RequestCount
RDS CPU & Connections	CPUUtilization, DatabaseConnections
Redis CPU & Memory	CPUUtilization, FreeableMemory
Lambda Invocations & Errors	monitor-worker + processor
SQS Queue Depth	main queue + DLQ

8.2 CloudWatch Alarms

Alarm	Threshold	Action
ECS CPU High	> 80%	SNS notification
ALB 5XX Spike	> 10 errors/min	SNS notification
Lambda Errors	> 0	SNS notification
DLQ Depth	> 0	SNS notification

8.3 AWS X-Ray Tracing

All three ECS services have X-Ray SDK integrated:

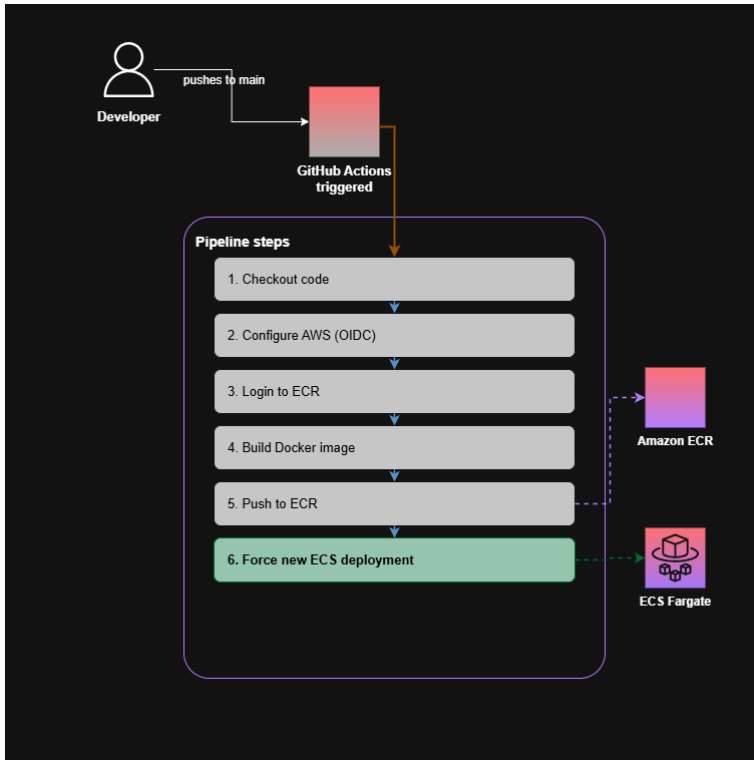
- `patch_all()` instruments all outgoing HTTP calls, SQLAlchemy queries
- Custom middleware records segment per request with HTTP status
- Traces visible in AWS X-Ray console for latency analysis

8.4 Structured Logging

- All Lambda functions use `print()` to CloudWatch Logs
- ECS services log to CloudWatch Log Groups via `awslogs` driver
- Log retention: 7 days (dev environment)

9. CI/CD Pipeline

9.1 Pipeline Overview



9.2 Environments

Branch	Environment	Auto-deploy
main	dev	Yes
prod (future)	prod	Yes (with approval)

9.3 Docker Build

Each service has its own Dockerfile:

- Base image: python:3.11-slim
- Dependencies installed via pip
- Non-root user for security
- Health check endpoint exposed

10. Infrastructure as Code

10.1 Terraform Module Structure

```
statusnest-infra/
|-- main.tf           # Root module, wires all modules together
|-- variables.tf      # environment variable
|-- outputs.tf        # Key resource outputs
|-- backend.tf        # S3 remote state (statusnest-terraform-state)
+-- modules/
    |-- vpc/          # VPC, subnets, IGW, NAT, route tables
    |-- rds/          # RDS PostgreSQL, SG, subnet group
    |-- elasticache/  # Redis cluster, SG, subnet group
    |-- ecr/          # ECR repositories (auth, monitor, status)
    |-- ecs/          # ECS cluster, task definitions, services, IAM
    |-- alb/          # ALB, target groups, listener rules
    |-- frontend/     # S3, CloudFront, OAC, bucket policy
    |-- waf/          # WAF v2 Web ACL (CloudFront-scoped)
    |-- waf-alb/      # WAF v2 Web ACL (ALB-scoped, regional)
+-- monitoring/      # CloudWatch dashboards + alarms
```

10.2 Remote State

Terraform state is stored in S3 with the following configuration:

Setting	Value
Bucket	statusnest-terraform-state
Region	us-east-1
Encryption	SSE-S3
Versioning	Enabled

10.3 Deployment

```
terraform init
terraform plan -var="environment=dev"
terraform apply -var="environment=dev"
```

11. Architectural Decisions (ADRs)

ADR-001: ECS Fargate over EC2

Decision: Use ECS Fargate for all API services.

Rationale: No EC2 instance management, automatic scaling, pay-per-task pricing, and better security posture (no SSH access needed).

Trade-off: Higher per-unit cost than EC2 at scale; cold starts on new task provisioning.

ADR-002: Lambda + SQS for Monitoring Engine

Decision: Use Lambda triggered by EventBridge, with SQS decoupling the processor.

Rationale: Monitoring is a periodic, stateless workload, ideal for serverless. SQS decouples ping results from database writes, adding resilience. A DLQ captures failed messages.

Trade-off: Cold starts on Lambda can add latency to the first ping after inactivity. Mitigated by EventBridge firing every 60s keeping Lambda warm.

ADR-003: Redis as Primary Status Cache

Decision: Redis (ElastiCache) is the primary data source for the public status page, with RDS as fallback.

Rationale: Status reads are extremely frequent (every page load). Redis provides sub-millisecond reads vs. RDS query latency. 90s TTL ensures freshness aligned to the 60s monitor interval.

Trade-off: Cache miss on Redis (e.g., first check or Lambda failure) falls back to RDS, which may return stale data.

ADR-004: CloudFront as Single Entry Point

Decision: All traffic (frontend + API) routes through a single CloudFront distribution.

Rationale: Eliminates CORS issues between frontend and API. Provides a single HTTPS endpoint. Enables WAF protection at the edge. Avoids mixed-content errors.

Trade-off: CloudFront caching must be carefully disabled for API paths (TTL=0 on /auth/* and /api/*).

ADR-005: PostgreSQL over Aurora

Decision: Use RDS PostgreSQL instead of Aurora.

Constraint: AWS account-level restrictions prevented Aurora provisioning.

Impact: No auto-scaling read replicas. Single-AZ in dev. Can be migrated to Aurora with minimal schema changes.

ADR-006: Soft Delete for Services

Decision: Service deletion sets is_active = false rather than deleting the row.

Rationale: Preserves historical service_status rows for that service. Prevents foreign key violations. Enables potential "restore" feature in the future.

Trade-off: Database grows over time with inactive records. Requires periodic archival job at scale.

ADR-007: OIDC over IAM Access Keys for CI/CD

Decision: GitHub Actions uses OIDC to assume an IAM role, with no static credentials.

Rationale: Static AWS access keys in GitHub secrets are a security risk. OIDC tokens are short-lived and scoped to specific repos and branches.

Trade-off: Slightly more complex initial setup (IAM role trust policy with GitHub OIDC provider).

ADR-008: pg8000 over psycopg2 for Lambda

Decision: Use pg8000 (pure Python) instead of psycopg2 in Lambda functions.

Rationale: psycopg2 requires native C binaries compiled for the Lambda Linux environment. pg8000 is pure Python and works without Docker build layers.

Trade-off: pg8000 is slightly slower than psycopg2 for high-throughput queries, acceptable for Lambda's low-frequency monitoring workload.

12. Trade-offs & Constraints

Area	Decision Made	What Was Traded Off
Database	RDS PostgreSQL (single-AZ)	No Aurora auto-scaling; no multi-AZ HA in dev
Domain	No custom domain (CloudFront default cert)	Less professional URL; no HTTPS with custom domain
Auth	Single-tenant (one hardcoded user)	No self-signup flow; not truly multi-tenant yet
Terraform coverage	Auth task def managed; monitor/status via CLI	Partial IaC drift
X-Ray	SDK integrated but daemon on 127.0.0.1	Traces may not reach X-Ray in all Fargate configs
Monitoring	Lambda pings from inside VPC	Cannot reach internal ALB endpoints (HTTP)
Cost	NAT Gateway running 24/7	~\$32/month fixed cost even at zero traffic

13. Scalability & Performance

13.1 Horizontal Scaling

Component	Scaling Mechanism
ECS Fargate	ECS Service Auto Scaling (target tracking on CPU/memory)
Lambda	Automatic concurrency scaling by AWS
RDS	Vertical scaling (instance size); read replicas for future
Redis	ElastiCache cluster mode (future)
CloudFront	Globally distributed, scales automatically

13.2 Performance Targets

Operation	Expected Latency
Status page load (Redis hit)	< 50ms API response
Status page load (RDS fallback)	< 200ms
Login / JWT issue	< 300ms

Operation	Expected Latency
Service ping (Lambda)	10s max timeout
SQS processing	< 5s end-to-end

13.3 Bottlenecks at Scale

- 1. RDS connections: ECS tasks each maintain a connection pool. At high task count, use RDS Proxy to pool connections.
- 2. Lambda concurrency: At 1000+ services, monitor Lambda may hit concurrency limits. Solution: paginate and fan out across multiple Lambda invocations.
- 3. SQS throughput: Standard queue supports unlimited throughput. Not a bottleneck.

14. Cost Considerations

14.1 Estimated Monthly Cost (Dev Environment)

Resource	Estimated Cost
ECS Fargate (3 tasks, 0.25 vCPU / 0.5GB each)	~\$15/month
RDS PostgreSQL (db.t3.micro)	~\$15/month
ElastiCache Redis (cache.t3.micro)	~\$15/month
NAT Gateway	~\$32/month
CloudFront (low traffic)	~\$1/month
Lambda (60s interval, ~43,800 invocations/month)	Free tier
SQS	Free tier
ALB	~\$16/month
Total	~\$94/month

14.2 Cost Optimization Opportunities

- NAT Gateway: Largest cost driver. In production, use VPC endpoints for AWS services (S3, SQS, ECR) to reduce NAT traffic.
- RDS: Move to Aurora Serverless v2 for auto-scaling to zero in low-traffic periods.
- ECS: Enable auto-scaling to scale down to 1 task during off-hours.
- Fargate Spot: Use Fargate Spot for non-critical services (status, monitor) to save up to 70%.

15. Known Limitations & Future Work

15.1 Known Limitations

Limitation	Impact	Mitigation
Single tenant (no signup)	Only one user (abood@statusnest.com)	Add registration flow
No custom domain	CloudFront URL only	Add Route 53 + ACM certificate
Monitor/status task defs not in Terraform	IaC drift	Import into Terraform
No email notifications	Subscribers stored but not emailed	Integrate SES + SNS
No HTTPS monitoring	Lambda uses HTTP internally	Use external ping endpoint
Redis TTL 90s	Status can be 90s stale on cache miss	Acceptable for MVP

15.2 Roadmap (Future Work)

Phase 2 — Multi-tenancy

- User self-registration flow
- Per-user isolated status pages
- Subdomain routing ({username}.statusnest.com)

Phase 3 — Notifications

- SES email alerts to subscribers on outage detection
- Slack / webhook integrations via SNS

Phase 4 — Production Hardening

- Multi-AZ RDS with automated backups
- Aurora Serverless v2
- RDS Proxy for connection pooling
- WAF rate limiting per IP
- Automated end-to-end tests in CI pipeline

Phase 5 — Analytics

- Uptime percentage calculations (30-day, 90-day)
- Response time trend graphs
- Incident postmortem templates

Appendix A — Technology Stack Summary

Layer	Technology	Version
Frontend	React	18
Frontend routing	React Router	v6
Frontend HTTP	Axios	latest
API framework	FastAPI	latest
ORM	SQLAlchemy	latest
Auth	python-jose + passlib	latest
DB driver (API)	psycopg2	latest
DB driver (Lambda)	pg8000	latest
Cache client	redis-py	latest
Container runtime	Docker	latest
IaC	Terraform	~> 5.0
CI/CD	GitHub Actions	—
Password hashing	bcrypt	4.0.1
Tracing	aws-xray-sdk	2.12.1

StatusNest — Solution Architecture Document v1.0
Muhammad Abdullah — github.com/aboodi679